SENTIMENT ANALYSIS ON TAMIL REGIONAL LANGUAGE USING MACHINE LEARNING ALGORITHMS

A PROJECT REPORT

Submitted by

NANDHINI G

RRN: 230282601085

Under the guidance of

Dr. A. JAYA

In the partial fulfillment for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

B.S. Abdur Rahman ™
Crescent
Institute of Science & Technology
Deemed to be University u/s 3 of the UGC Act, 1956

MAY 2025

BONAFIDE CERTIFICATE

Certified that this project report titled "**SENTIMENT ANALYSIS ON TAMIL REGIONAL LANGUAGE USING MACHINE LEARNING ALGORTHIMS"** is the bonafide work of **Ms. NANDHINI G (RRN: 230282601085)** who carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here in does not from part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. A. JAYA
SUPERVISOR
Professor & Director (CDOE)
Department of Computer Applications,
B.S. Abdur Rahman Crescent Institute of Science and Technology,
Vandalur, Chennai-600048.

SIGNATURE

Dr. M. SYED MASOOD
HEAD OF THE DEPARTMENT
Associate Professor
Department of Computer Applications,
B.S. Abdur Rahman Crescent Institute of Science and Technology,
Vandalur, Chennai-600048.

B.S. Abdur Rahman ™
Crescent
Institute of Science & Technology
Deemed to be University u/s 3 of the UGC Act, 1956

VIVA-VOCE EXAMINATION

The viva-voce examination of the project work titled **"SENTIMENT ANALYSIS ON TAMIL REGIONAL LANGUAGE USING MACHINE LEARNING ALGORITHMS"** submitted by **Ms. NANDHINI G, RRN: 230282601085,** is held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

I thank the **Almighty** for showering his blessings upon me in completing the project. I submit this project with a deep sense of gratitude and reverence for my beloved parents for their moral support and encouragements.

I sincerely express my heartfelt gratitude to **Dr. T. Murugesan**, **Vice Chancellor**, B.S. Abdur Rahman Crescent Institute of Science and Technology and **Dr. N. Raja Hussain , Registrar** for furnishing every essential facility for doing my project.

I owe my sincere gratitude to **Dr. Sharmila Sankar, Professor and Dean,** School of Computer, Information and Mathematical Sciences and I also thank **Dr. M. Syed Masood, Associate Professor and Head**, Department of Computer Applications for providing strong oversight of vision, strategic direction, encouragement and valuable suggestions in completing my project work.

I convey my earnest thanks to my project guide **Dr. A. JAYA, Professor & Director(CDOE),** Department of Computer Applications, for her valueable guidance and support through out the project.

I sincerely extend my gratitude to our Project Committee Members, **Dr. A. Abdul Azeez Khan, Associate Professor**, **Dr. K. Javubar Sathick, Associate Professor** and our Project Coordinators, **Dr. A. K. Ashfauk Ahamed, Assistant Professor (Sr. Grade)**, **Mr. M. Mohammed Jalaluddin, Assistant Professor**, Department of Computer Applications, for their invaluable insights, constructive feedback, and continuous support throughout our project.

I extend my sincere thanks to all my faculty members for their valuable suggestions, timely advice and support to complete the project.

- NANDHINI.G

ABSTRACT

Social media platforms have emerged as powerful tools for individuals to express their thoughts, opinions, and emotions. Among them, YouTube plays a prominent role where users interact and provide feedback through comments on various content. This project focuses on analysing Tamil-language comments using sentiment analysis techniques. Tamil, being a widely spoken regional language with a complex grammatical structure and diverse dialects, poses challenges for traditional sentiment analysis models. Therefore, this project aims to build an efficient system for identifying the emotional tone positive, negative, or neutral of Tamil comments on social media platforms using machine learning algorithms.

This project is structured into five major modules: Data Collection Module, Preprocessing Module, Model Training & Evaluation Module, and Performance Analysis Module. In the Data Collection Module, Tamil comments are collected from platforms like YouTube. The Preprocessing Module handles tasks like cleaning, normalization, and removing irrelevant information. The Model Training & Evaluation Module involves training four machine learning algorithms Support Vector Machine (SVM), Logistic Regression, Random Forest, and Naïve Bayes on the labeled data. Finally, the Performance Analysis Module compares each model using standard metrics such as accuracy, precision, recall, and F1-score to determine the best-performing model for Tamil sentiment classification.

The highlight of this project lies in its focus on regional language sentiment analysis tailored for Tamil. The project is developed using Python for machine learning implementation, and HTML, CSS, and Python for front-end and back-end integration. Through accurate sentiment analysis, this system helps content creators and organizations make informed, data-driven decisions based on real user emotions and feedback, ultimately enhancing user engagement and satisfaction.

TABLE OF CONTENT

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	V
	LIST OF TABLES	Ix
	LIST OF FIGURES	X
	LIST OF ABBREVIATIONS	Xi
1	**INTRODUCTION**	1
	1.1 GENERAL	1
	1.2 EXISITING SYSTEM	2
	1.2.1 Literature Survey	3
	1.2.2 Disadvantages of Existing System	7
	1.3 PROPOSED SYSTEM	8
	1.3.1 Advantages of proposed system	8
	1.4 ORGANIZATION OF CHAPTERS	9
2	**PROBLEM DEFINITION AND METHODOLOGY**	10
	2.1 PROBLEM DEFINITION	10
	2.2 METHODOLOGY	11
3	**DEVELOPMENT PROCESS**	13
	3.1 REQUIREMENT ANALYSIS	13
	3.1.1 Input Requirements	13
	3.1.2 Output Requirements	13
	3.1.3 Resource Requirements	14
	3.1.3.1 Hardware Requirements	14
	3.1.3.2 Software Requirement	14

3.2 SYSTEM DESIGN 15
3.2.1 System Architecture 15
3.2.2 Data Flow Diagram 17
3.3 IMPLEMENTATION 18
3.3.1 Technologies Used 18
3.3.1.2 Algorithm for Sentiment Analysis 19
3.3.2 Sentiment Analysis Module 20
3.3.2.1 Data Collection 20
3.3.2.2 Data Pre-Processing 20
3.3.2.3 Feature Extraction 21
3.3.2.4 Training and Testing Data 21
3.3.2.5 Sentiment Analysis 22
3.3.2.5 Sentiment Classification 24
3.3.2.7 Sentiment Orientation 24
3.4 TESTING 25
3.4.1 Unit Testing 25
3.4.2 Integration Testing 26
3.4.3 Function Testing 27
4 RESULT ANALYSIS AND DISCUSSION 28
4.1 RESULT ANALYSIS 28
4.2 OUTPUT OF THIS PROJECT 28
5 CONCLUSION AND FUTURE ENHANCEMENT 36
5.1 CONCLUSION 36
5.2 FUTURE ENHANCEMENT 37
REFERENCES 38
APPENDIX 44

CERTIFICATIONS 46

TECHNICAL BIOGRAPHY 49

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
3.1	Unit Test Cases for Preprocessing and Model	25
3.2	Integration Test Cases for Full pipeline	26
3.3	Functional Test Cases	27

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
2.1	Hierarchical Diagram for Sentiment Analysis	12
3.1	Overall System Architecture of Sentiment Analysis System	15
3.2	Data Flow Diagram of sentiment analysis system	17
3.4	Screenshot of bar plot	23
3.5	Screenshot of confusion matrix	25
4.1	Screenshot of Home page	29
4.2	Screenshot of Upload page	30
4.3	Screenshot of Select Comment page	31
4.4	Screenshot of Report page	31
4.5	Screenshot of Predict page	32
4.6	Screenshot of Predict Analysis page	33
4.7	Screenshot of About page	34
4.8	Screenshot of Feedback page	35

LIST OF ABBREVATIONS

API – Application Programming Interface

CPU – Central Processing Unit

CSS – Cascading Style Sheets

NLP – Natural Language Processing

ML – Machine Learning

SVM – Support Vector Machine

RF – Random Forest

LR – Logistic Regression

NB – Naive Bayes

TF-IDF – Term Frequency-Inverse Document Frequency

CSV – Comma-Separated Values

UI – User Interface

SNA – Social Network Analysis

POS – Positive Sentiment

NEG – Negative Sentiment

NEU – Neutral Sentiment

URL – Uniform Resource Locator

HTML – Hyper Text Markup Language

RAM – Random Access Memor

CHAPTER 1

INTRODUCTION

This chapter deals with the general introduction of the project, existing system and the proposed system for this project. Existing system elucidates the current system and its limitations and proposed system provides a solution to overcome those limitations.

1.1 GENERAL

As social media continues to shape digital interaction, platforms like YouTube have become key spaces where individuals express their opinions, feelings, and reactions through comments and reviews. These user-generated comments provide a valuable dataset for analyzing public sentiment. While sentiment analysis has advanced for global languages such as English, regional languages like Tamil remain less explored due to their linguistic complexity. Tamil, a classical language spoken by over 80 million people, involves unique grammatical structures, dialect variations, and informal usage, which makes sentiment classification more challenging.

The main purpose of this project is to build a sentiment analysis system specifically for the Tamil language using machine learning techniques. The process begins with collecting Tamil-language comments from social media platforms. These comments are then cleaned and pre processed to remove noise, normalize text, and tokenize words. The cleaned data is manually labeled as positive, negative, or neutral to form a training dataset. Machine learning algorithms including Support Vector Machine (SVM), Logistic Regression, Random Forest, and Naïve Bayes are implemented and compared. TF-IDF vectorization is used to convert the text into numerical form for model training and evaluation.

Sentiment analysis plays a crucial role in understanding user behavior, feedback, and satisfaction in the digital world. The goal of this project is to contribute towards natural language processing for regional languages by

building a reliable sentiment analysis tool for Tamil. It offers a practical solution for analyzing user feedback on digital platforms, helping content creators and organizations make informed decisions. This system also opens up future possibilities for expanding into deep learning, browser-based tools, and multilingual sentiment analysis platforms.

1.2 EXISTING SYSTEM

Most existing systems for sentiment analysis are designed for widely spoken languages like English and a few African languages such as Swahili and Yoruba. These systems use multilingual models like mBERT and FastText, which are trained on large amounts of global data to analyze text from sources like social media and reviews.

However, these systems are not built specifically for low-resource languages like Tamil. They face difficulty in understanding the context when regional slang, local expressions, or dialect-based grammar are used. As a result, the output can often be inaccurate or unrelated to the actual meaning of the content.

A major challenge is the lack of annotated datasets for Tamil and its regional dialects. Tamil comments often include a mix of local phrases and code-switching, which these general models cannot interpret correctly. This limits their ability to identify the real sentiment behind the text.

The absence of proper language-specific preprocessing tools and reliance on unrelated training data further affects the performance. So even though progress has been made in multilingual sentiment analysis, current systems are not well-suited for capturing the cultural and linguistic richness of Tamil language content.

1.2.1 Literature Survey

This section explores the various, survey of the research articles of based on Sentiment Analysis.

Title: Improving Sentiment Analysis Using Text Network Features Within Different Machine Learning Algorithms

Authors: Ali Mohamed Alnasrawi, Asia Mahdi NaserAlzubaidi.

This study investigates student feedback on e-learning using a hybrid BiLSTM- fuzzy model, This study enhances sentiment analysis by integrating text network analysis with various machine learning algorithms. Text from Yelp reviews is converted into a network of word co-occurrences, and centrality measures (like eigenvector, closeness, betweenness, etc.) are used to extract features. These are combined with traditional features for classification. Ten ML algorithms were tested (e.g., SVM, logistic regression, random forest, neural networks), with the neural network model achieving the best performance with 77 %.This approach proves that incorporating network-based features can significantly boost sentiment classification and offers a more insightful method for analyzing textual data.

Title: Afri-Senti: A Twitter Sentiment Analysis Benchmark for African Languages Analysis

Authors: Shamsuddeen Hassan Muhammad, IdrisAbdulmumin,
Abinew Ali Ayele, NedjmaOusidhoum.

This paper introduces AfriSenti, the largest multilingual sentiment analysis benchmark for African languages, consisting of 110,000+ tweets in 14 languages (including Hausa, Swahili, Yorùbá, Amharic, and more). Tweets were manually annotated by native speakers with sentiment labels: positive, negative, or neutral. The team conducted baseline experiments using pre-trained models like XLM-R, AfroXLMR, and AfriBERTa, with the best performance (F1-score up to 71.2%) achieved by AfroXLMR-large.The study addresses challenges like low-resource language processing, code-switching, and script variation, and provides open-source datasets and lexicons to support further research in African NLP.

Title: A Review on Sentiment Analysis from Social Media Platform.

Authors: Margarita Rodríguez-Ibáñez, Antonio Casánez-Ventura,

Félix Castejón- Mateos, Pedro-Manuel Cuenca-Jiménez.

This comprehensive review explores sentiment analysis techniques and applications on social media, especially Twitter. It covers traditional machine learning methods (like SVMs and lexicons) as well as advanced models like Transformers (BERT, GPT), although noting limited adoption due to resource requirements. The paper emphasizes temporal sentiment analysis and causality (Granger's causality) in domains like politics, health, cyberbullying, and stock prediction. With over 8,000 patents and 2,300 academic articles reviewed, it shows strong industry interest and ongoing research opportunities in this space.

Title: Prediction and Analysis of Indonesia Presidential Election from Twitter Using Sentiment Analysis

Authors: WidodoBudiharto & Meiliana

This study predicts the outcome of Indonesia's 2019 Presidential election using sentiment analysis on Twitter data. Tweets related to Jokowi and Prabowo were collected and preprocessed using R. A sentiment scoring algorithm was used to classify opinions as positive, negative, or neutral. Based on tweet polarity, Jokowi showed higher public support, which aligned with offline survey results. The framework includes OAuth-based tweet collection, tweet cleaning, keyword extraction, and sentiment calculation, demonstrating how social media can act as an alternative election prediction tool.

Title: Artificial Intelligence and Sentiment Analysis: A Review in Competitive Research

Authors: M.HamedTaherdoost & MitraMadanchian.

This scoping review (2012–2022) explores how AI-based sentiment analysis supports competitive business research. It emphasizes AI techniques such as machine learning, deep learning, and NLP for evaluating consumer opinions on brands, services, and competitors. Applications include customer behavior analysis, product positioning, and market prediction. The paper categorizes methods (lexicon-based vs. AI-based), highlights challenges like

preprocessing and domain dependency, and shows rising research interest particularly from 2019 onward. The study underscores sentiment analysis as a vital tool for strategic business decisions.

Title: AI-Driven Sentiment Analysis in Social Media Content

Authors: VasylBuhas, IhorPonomarenko, Oksana Kazak, NataliiaKorshun.

This paper presents a detailed exploration of how AI-driven sentiment analysis enhances digital marketing by analyzing user sentiments in text, emojis, and visual content on social media. It reviews various techniques lexicon-based, statistical, machine learning, deep learning, and hybrid approaches for identifying consumer emotions. The study emphasizes the use of advanced methods like BERT, GPT, emoji embeddings, and transfer learning to detect nuanced expressions including sarcasm and cultural context. By leveraging tools like One-Hot Encoding, custom embeddings, and real-time monitoring, the approach significantly boosts accuracy in understanding user behavior and optimizing marketing strategies, especially for Gen Z and Alpha audiences.

Title: Detecting Sarcastic Sentiment in Social Media Posts: A Machine Learning Framework for the Aviation Sector

Authors: Abdul-MananIddrisu, Solomon Mensah, Fredrick Boafo, Govindha R. Yeluripati, Patrick Kudjo.

This paper proposes a novel machine learning framework for identifying sarcastic versus non-sarcastic sentiments in social media posts, specifically within the aviation industry. Recognizing sarcasm is crucial, as it often conveys dissatisfaction in a subtle manner. The authors introduce a three-stage framework Assemble Deft, Edify Authenticate, and Forecast designed to classify opinions using Recurrent Neural Networks (RNN) with Gated Recurrent Units (GRU) and Support Vector Machines (SVM). The model is trained on tweets about the impact of COVID-19 on air travel. Their results show that optimized SVM outperforms RNNs, offering improved precision, accuracy, recall, and F1-scores. The research highlights the importance

of accounting for sarcasm to better interpret customer feedback and improve airline service strategies.

Title: Hybrid Machine Learning Model for Turkish Tweet Sentiment and Satisfaction Analysis
Authors: Abdul fattah Ba Alawi, Ferhat Bozkurt.

This research introduces a hybrid sentiment analysis model focused on Turkish tweets about university satisfaction. The model combines BERT, Bi LSTM, and triple-branch CNN architectures to handle Turkish's rich morphology. Trained on a manually labeled dataset of 17,793 tweets, the hybrid BERT-Bi LSTM-CNN model achieved over 91% accuracy and an F1 score of 88%, outperforming traditional ML and DL models. The study emphasizes the effectiveness of hybrid deep learning in handling complex languages and provides valuable insights for educational institutions aiming to improve public engagement and service quality based on social media sentiment.

Title: Transformer-Based Sentiment Analysis for Bengali: Overcoming Language Limitations
Authors: Md. NesarulHoque, Umme Salma, Md. Jamal Uddin, Md. MartuzaAhamad, SakifaAktar.

This study explores sentiment analysis for Bengali, a low-resource language, using advanced transformer models. The authors evaluate five models mBERT, BanglaBERT, Bangla-Bert-Base, DistilmBERT, and XLM-R on a dataset of 11,807 Bengali YouTube drama reviews. To improve performance, they introduce a Transformer-Ensemble method that combines predictions from all five models. The proposed ensemble achieves a standout accuracy of 95.97% and an F1-score of 95.96%, outperforming previous Bengali SA research. This work highlights the growing potential of multilingual and language-specific transformer models in analyzing underrepresented languages and emphasizes the importance of enriched datasets for better sentiment classification.

Title: Challenges and Opportunities in Text-Based Emotion Detection: A Comprehensive Survey

Authors: Abdullah Al Maruf, FahimaKhanam, Md. MahmudulHaque, ZakariaMasudJiyad, M. F. Mridha, ZeyarAung

This survey provides a deep dive into the emerging field of text-based emotion detection (TBED), highlighting its applications in mental health, marketing, politics, and social media. It reviews machine learning and deep learning methods, including lexicon-based, transformer-based, and hybrid models. It also outlines key challenges such as multilingual complexity, emotion ambiguity, and scalability, while suggesting promising directions like cross-lingual transfer learning and emotion intensity modeling. This work serves as a valuable reference for both beginners and experts exploring emotion AI systems.

1.2.2 Disadvantages Of Existing System

The following are the disadvantages of existing system

- Time-consuming.
- Subjectivity.
- Inconsistent Insights.
- Limited Scalability.
- Lack of Real-Time Analysis.

1.3 PROPOSED SYSTEM

The proposed system for sentiment analysis on Tamil regional language content leverages advanced Natural Language Processing (NLP) and machine learning techniques to accurately identify and interpret the emotional tone of text data. Specifically designed to handle the linguistic richness and complexity of Tamil, this system incorporates customized preprocessing methods such as text cleaning, tokenization, and normalization suited for Tamil grammar, vocabulary, and dialect variations.

The model uses algorithms like Support Vector Machine (SVM), Random Forest, and Logistic Regression, which are trained on annotated Tamil datasets collected from platforms like social media, review sites, and public forums. The system classifies feedback into positive, negative, or neutral categories, enabling real-time sentiment tracking and trend analysis.

By addressing the limitations of generalized models and focusing on dialect-sensitive processing, the system enhances accuracy, reduces misclassification, and ensures contextual relevance. It is scalable, allowing future integration of other low-resource languages and broader regional coverage.

Overall, this approach offers a reliable, efficient, and culturally aware solution for analyzing Tamil sentiment, helping developers educators, and researchers to gain deeper insights into user emotions and feedback in a regional contex.

1.3.1 Advantages of Proposed System

The following are the advantages of proposed system

- Scalability.
- Efficiency and Consistency.
- Data-Driven Decisions.
- Improved Accuracy.
- Real-Time Sentiment Detection.
- Culturally Relevant Insights.

1.4 ORGANIZATION OF THE CHAPTERS

Organization of the chapter is the short summary of the forth coming chapters. In this, Chapter 1 deals with the introduction, existing system, and the proposed system of the project were discussed. Chapter 2 deals with problem definition and methodology. Chapter 3 deals with the development process, the requirement analysis, system design implementation and testing of the system are discussed. The results of the project are analysed and concluded in the chapter 4.

CHAPTER 2

PROBLEM DEFINITION AND METHODOLOGY

In this previous chapter, the existing and the proposed system of the project are discussed. This chapter deals with the problem definition and methodology. The problem definition discusses about the objective of the project and the methodology used to develop the project.

2.1 PROBLEM DEFINITION

The main objective of this project is to develop a sentiment analysis system for Tamil regional language using machine learning algorithms. Tamil, being a morphologically rich and diverse language with many dialects, poses unique challenges for traditional sentiment analysis systems. The proposed system aims to overcome these challenges by building a model that understands Tamil vocabulary, grammar patterns, and common usage including code-mixed text.

The system collects user-generated Tamil text data from various platforms such as social media and forums. After undergoing customized preprocessing suitable for Tamil language (like text normalization, tokenization, and cleaning), machine learning algorithms such as Support Vector Machine (SVM), Logistic Regression, and Random Forest are trained on the labeled dataset. These models classify the sentiment of the input as positive, negative, or neutral.

This project aims to assist researchers, educators, and developers in understanding public sentiment expressed in Tamil content and to support decision-making in regional communication, education, media, and business applications.

To enhance the system's effectiveness, the project also explores feature extraction techniques such as TF-IDF and word embeddings, which help in capturing the semantic meaning of Tamil words more accurately. These techniques play a crucial role in improving the model's ability to detect subtle

differences in sentiment, especially in complex or code-mixed Tamil expressions.

2.2 METHODOLOGY

Methodology refers to the techniques and procedures used to analyze information in an organized manner. This project follows a top-down approach, ensuring a comprehensive understanding of the system. The proposed system is divided into five key modules: Data Collection, Preprocessing, Feature Extraction, Model Training and Testing, and Sentiment Analysis. In the Data Collection module, Tamil regional language comments are gathered from various sources such as social media platforms, news websites, and review sections. The Preprocessing module cleans and prepares the collected data by removing unwanted symbols, handling mixed-language text, and normalizing the Tamil script to ensure consistency. The Feature Extraction module transforms the cleaned text into a machine-readable format using techniques like TF-IDF and Count Vectorizer, enabling the system to detect meaningful patterns. The Model Training and Testing module involves applying machine learning algorithms such as Logistic Regression, SVM, and Random Forest to classify sentiments into positive, negative, or neutral categories.

These models are trained on labeled datasets and tested to evaluate their accuracy and reliability. Finally, the Sentiment Analysis module integrates these models into a user-friendly platform, enabling the real-time prediction of sentiment in Tamil feedback and comments. The project is developed using Python for data handling and machine learning, while HTML, CSS are used for building and managing the web interface.

To boost performance, the system includes model fine-tuning and error analysis using metrics like precision and F1-score. It also addresses data imbalance through oversampling or class weighting. These steps ensure improved accuracy.

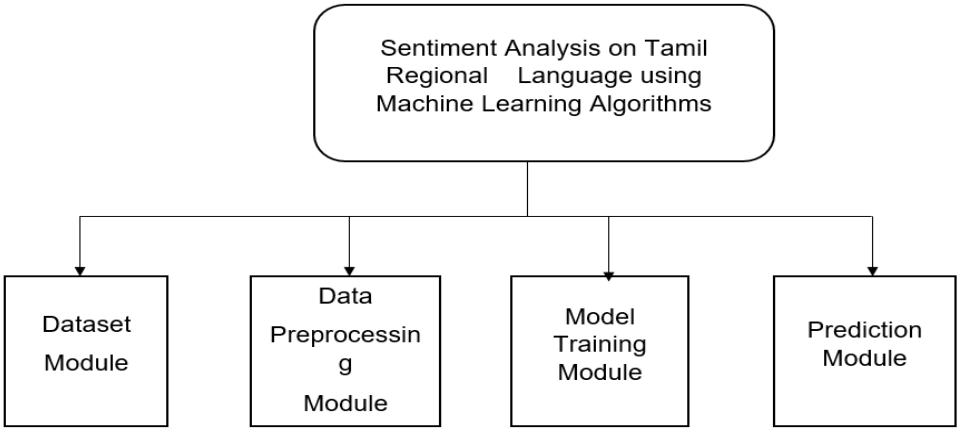


Figure 2.1 Hierarchical Diagram for Sentiment Analysis

Figure 2.1 depicts the hierarchical architecture of the project "Sentiment Analysis on Tamil Regional Language using Machine Learning Algorithms." The system begins with data collection and flows through preprocessing, feature extraction, model training, and sentiment prediction.

The core of the system lies in its language-aware preprocessing techniques and region-specific model training, ensuring that dialectal variations are respected and properly analyzed. By tailoring machine learning algorithms to Tamil language nuances, the system delivers more accurate and context-aware sentiment classifications, making it more effective than generalized models.

Chapter 2 discussed about problem definition and methodology. The next chapter 3 deals with the development process of the project. It explains about the requirement analysis, design phase and implementation.

CHAPTER 3

DEVELOPMENT PROCESS

In Chapter 2, the problem definition and the methodology were discussed in detail. This chapter focuses on the overall design and development of the project. It explains the architectural design, detailed design, implementation, and testing phases clearly.

The development process is a vital part of the software development life cycle, as it translates requirements into an actual functioning system. This phase involves analyzing both product and resource requirements to ensure successful implementation.

3.1 REQUIREMENT ANALYSIS

The Requirement Analysis section outlines the input, output, and resource requirements for the project. These requirements are essential for the successful execution and deployment of the sentiment analysis system.

3.1.1 Input Requirements

The input to this project consists of user-provided text data such as Tamil regional comments, reviews, or feedback collected from various platforms like social media, YouTube, and other forums. The system accepts raw Tamil textual content, which may include slang, colloquial expressions, or dialectal variations specific to regions such as Kongu Tamil and Madurai Tamil.

3.1.2 Output Requirements

The output of the system is the sentiment classification of the input Tamil text. Each input comment is categorized into one of three sentiment classes: Positive, Negative, or Neutral. The output also includes sentiment reports and visualizations such as pie charts or word clouds that help in understanding the emotional trends in Tamil regional feedback.

3.1.3 Resource Requirements

To build and run the sentiment analysis system efficiently, specific hardware and software resources are required. The configuration details are provided below.

3.1.3.1 Hardware Requirements

The following hardware configuration is required:

- Device: Laptop or Desktop Computer.
- Processor: Intel Core i5 or higher (or equivalent AMD processor).
- RAM: Minimum 8 GB.
- Storage: 500GB/1TB.
- Network: Internet connectivity.

3.1.3.2 Software Requirements

The following are the software and libraries used:

- Front-End: HTML, CSS.
- Back-End: Python.
- Operating System: Windows 10 .
- Development Tools: Visual Studio Code.

Frameworks and Libraries:

- Flask – Lightweight backend web server
- Pandas &NumPy – Data preprocessing and manipulation
- scikit-learn – Machine Learning algorithms (e.g., SVM, Naive Bayes, Logistic Regression)
- seaborn – Visualization of accuracy and confusion matrix
- re – Text cleaning using Regular Expressions
- joblib – Model serialization

3.2 SYSTEM DESIGN

The design explains the various modules of the project, the overall architecture of the project, workflows and data flows of the project etc. Systems design is the process of defining the architecture, modules, interfaces, and data for a system to satisfy specified requirements. Systems design could be seen as the application of systems theory to product development. There is some overlap with the disciplines of systems analysis, systems architecture and systems engineering.

3.2.1 System Architecture

This architecture will give explanations about the entire concept about the project. This defines the structure of the developed system, which comprises different modules and their externally visible properties and the relationship Among system and The Figure 3.1 shows complete architecture for this project.

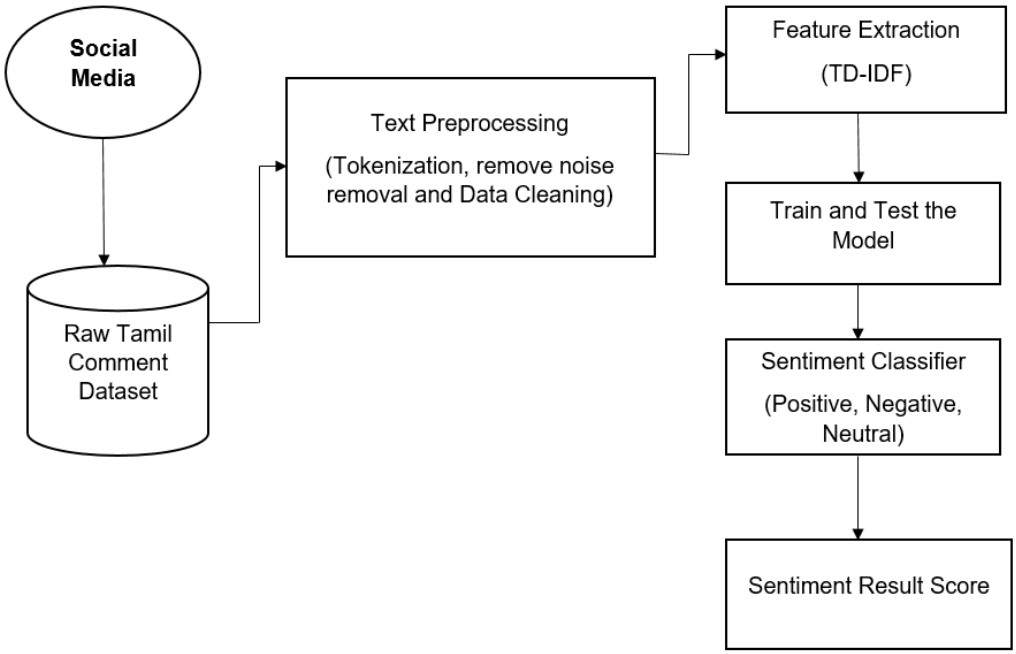


Figure 3.1 Overall System Architecture of Sentiment Analysis System

The above Figure 3.1 illustrates the architecture of the proposed system titled “Sentiment Analysis on Tamil Regional Language Using Machine Learning Algorithms”, which provides a complete end-to-end process—from data collection to sentiment classification and result deployment.

The system starts with the data collection phase, where raw Tamil textual data is gathered from various sources such as social media platforms and online forums. This data is often informal and unstructured, making it necessary to clean and preprocess before analysis.

Once collected, the data enters the Text Preprocessing Module, which performs crucial tasks such as tokenization, stemming, lemmatization, and data cleaning. Tokenization breaks the text into individual words or tokens for easier processing. Stemming and lemmatization standardize words to their root forms, reducing noise and improving consistency. Data cleaning eliminates irrelevant content such as emojis, URLs, and special characters, ensuring the dataset is clean and usable.

The cleaned data is then forwarded to the Feature Extraction Module, where techniques such as TF-IDF (Term Frequency-Inverse Document Frequency) are applied. These techniques transform textual data into meaningful numerical representations that preserve the semantic structure of the original text, making it suitable for machine learning.

In the Machine Learning Module, these feature vectors are used to train multiple classification algorithms including Logistic Regression, Support Vector Machine (SVM), and Random Forest. Each model is evaluated, and the best-performing one is selected based on its accuracy and efficiency in classifying sentiments. This phase is critical to building a robust and intelligent sentiment prediction engine.

Once training is complete, the system is ready to classify new Tamil text inputs into three sentiment categories: positive, neutral, or negative. This classification is performed by the Sentiment Classifier, which acts as the decision-making unit of the architecture.

The classified results are then stored in the Sentiment Result Store, enabling further reporting, visualization, and integration with other platforms. These outputs can be used for real-time sentiment tracking, social media monitoring, or customer feedback analysis.

3.2.2 Data Flow Diagram

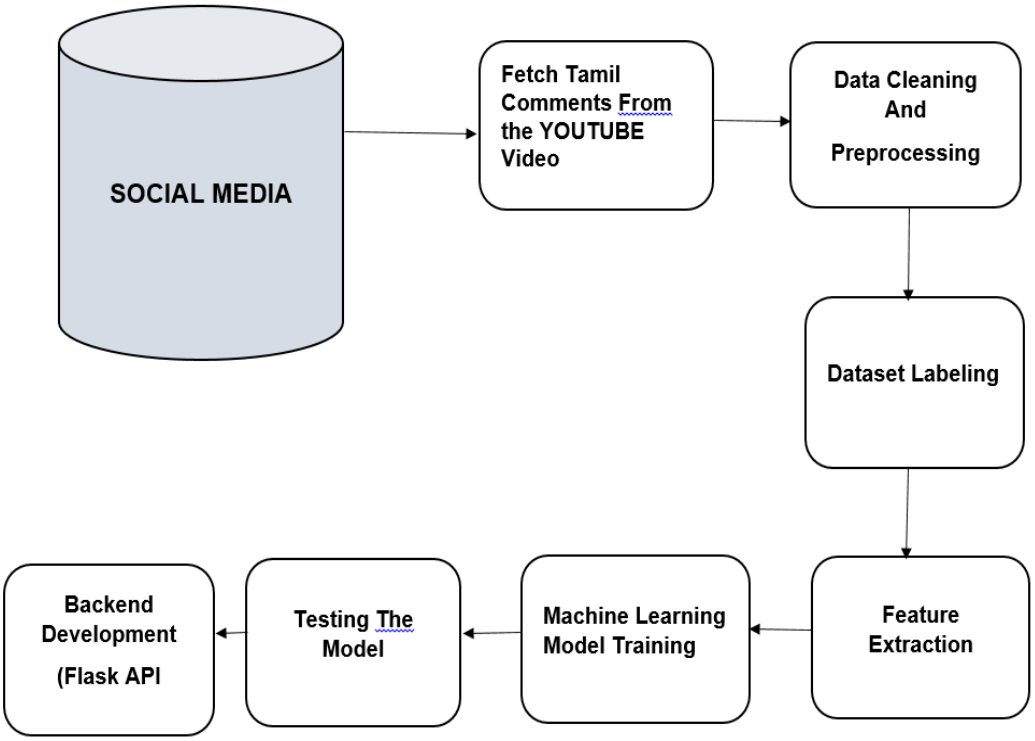


Figure 3.2 Data flow Diagram of Sentiment Analysis System

The Figure 3.2 Data flow diagram illustrates the end-to-end process of performing sentiment analysis on Tamil comments fetched from YouTube. It begins with collecting comments from social media platforms, specifically YouTube videos. These raw comments are then cleaned and preprocessed to remove noise and standardize the data. The cleaned data is labeled based on sentiment positive, negative, or neutral and then passed through a feature extraction step, where text data is converted into numerical form using methods like TF-IDF. This prepared dataset is used to train various machine learning models. Once the model is trained, it is tested for accuracy and performance. Finally, the tested model is integrated into a Flask-based backend, allowing real-time sentiment prediction through a web interface.

3.3 IMPLEMENTATION

Implementation includes all those activities that take place to convert fro m old system to the new. The new system may be totally new, replacing an existing system or it may be major modification to the system currently put into use. In this project, design is done by using the Python.

3.3.1 Technologies Used

The front-end for the sentiment analysis platform has been developed using HTML and CSS for structure and styling. The back-end is built using Flask, a Python-based web framework, which connects the interface with the machine learning model. JSON is used to exchange data between the front-end and back-end, and the sentiment analysis model is developed in Python using the Logistic Regression algorithm.

HTML stands for Hyper Text Markup Language. It is the standard markup language used to design web pages and applications. HTML is made up of tags and attributes that define the structure of web content.

CSS stands for Cascading Style Sheets. It is used to define the visual appearance of HTML documents. CSS controls aspects like color, font, layout, and spacing, helping to create a clean and responsive design.

JSON stands for JavaScript Object Notation. It is a lightweight data format used to exchange data between the server and the client. JSON is easy to read and write, making it suitable for APIs and data transfer.

Flask is a lightweight web framework written in Python. It is used to build the back-end part of the application and manage routes, APIs, and server-side logic. Flask helps connect the user interface to the machine learning model.

Python is a high-level programming language used for various purposes, including web development, data analysis, and machine learning. In this project, Python is used to implement the sentiment analysis logics.

3.3.1.2 Algorithm for Sentiment Analysis

In this project, multiple supervised machine learning algorithms are used to classify sentiment in Tamil text data, which often includes code-mixing, dialect variations, and morphological richness. The goal is to compare the performance of different models and find the best fit for sentiment classification in a low-resource language setting. The algorithms used include Naïve Bayes, SVM, Logistic Regression, and Random Forest, trained on vectorized Tamil data using Count Vectorizer and TF-IDF, and evaluated on metrics like accuracy, precision, recall, and F1-score.

- Naïve Bayes Classifier

Naïve Bayes is a simple and fast algorithm based on Bayes' Theorem. It assumes independence between features and works well for text data. In this project, it classifies Tamil comments into positive, negative, or neutral based on word frequency. It's lightweight and effective for initial sentiment analysis in large datasets.

- Support Vector Machine (SVM)

SVM is a powerful model that finds the best boundary between classes. It performs well in high-dimensional, sparse data like text. In this project, SVM handles Tamil-English mixed comments and is useful for managing noisy and culturally rich social media data. Its ability to generalize makes it a reliable classifier.

- Logistic Regression

Logistic Regression is a simple yet effective model for classification. It’s interpretable, fast, and works well for linearly separable data. In this project, it serves as a baseline model, helping to measure the improvement from more advanced algorithms.

- Random Forest

Random Forest is an ensemble model that combines multiple decision trees for better accuracy and stability. It handles noisy, inconsistent Tamil data well and captures complex patterns. It also provides feature importance, helping understand which words affect sentiment most. Ideal for unstructured and imbalanced datasets.

3.3.2 Sentiment Analysis Module

The sentiment analysis module focuses on identifying the emotional tone behind textual data, particularly user comments in Tamil. This module involves several essential steps, starting from collecting raw text data to processing and analyzing it using machine learning techniques. The goal is to classify each comment as Positive, Negative, or Neutral, thereby gaining insights into public opinion. The entire process includes data collection, cleaning, feature extraction, and model training using multiple classification algorithms.

3.3.2.1 Data Collection

The first and most fundamental step in the implementation was data collection. Since there is a lack of widely available Tamil sentiment datasets, a hybrid approach was followed to build the corpus. The data was collected manually and programmatically from various online platforms such as YouTube, and other social media platforms. The collected data consisted of user-generated comments in Tamil written in Unicode format.

3.3.2.2 Data Pre Processing

Once the raw text was gathered, preprocessing was initiated. This step ensured that the data was clean, uniform, and machine-readable. Preprocessing included the following operations:

- Noise Removal: Special characters, emojis, hyperlinks, punctuation marks, and hashtags were removed using regular expressions.

- Lowercasing: All comments were converted to lowercase to maintain consistency.
- Stop word Removal: Common but contextually irrelevant Tamil stop words were removed to reduce noise in feature extraction.
- Tokenization: Sentences were split into words or tokens using space-based or regex-based tokenizers.

3.3.2.3 Feature Extraction

Feature extraction is one of the most critical aspects of text classification. The goal is to convert human language into a format understandable by machine learning algorithms. one prominent feature extraction methods were explored:

- TF-IDF (Term Frequency-Inverse Document Frequency): A more sophisticated method that considers both the frequency of a term in a document and its frequency across all documents, thereby penalizing commonly used words and highlighting rare but significant terms.
- Both vectorizers were tested, and their outputs were used in the model training process to identify the best-performing combination.

3.3.2.4 Training and Testing Data

Following preprocessing, the dataset was labeled manually with sentiment tags: Positive, Negative, or Neutral. This labeled dataset was split into training (80%) and testing (20%) subsets using stratified sampling to maintain class balance. Vectorization techniques like Count Vectorizer and TF-IDF were used to convert textual comments into numerical form. These techniques created a high-dimensional feature space where each word was treated as a feature. The result was a structured numeric dataset, which could be fed into machine learning models.

Four classification algorithms were trained on this dataset:

- Logistic Regression
- Support Vector Machine (SVM)

- Random Forest Classifier
- Naïve Bayes Classifier
- Each model was evaluated using performance metrics such as Accuracy, Precision, Recall, and F1 Score.

3.3.2.5 Sentiment Analysis

In this project, sentiment analysis results on Tamil regional language content are visualized using bar charts to provide a clear and structured understanding of user opinions. After the text data is classified into positive, negative, and neutral sentiments, the results are displayed through bar graphs to show the distribution of sentiment across various categories.

Bar charts are used to represent the number of comments under each sentiment type for specific topics, such as Tamil movie reviews, social media posts, or regional news discussions. This allows viewers to easily compare the volume of different sentiments across multiple data groups.

By visualizing sentiment classification through bar charts, it becomes easier to identify patterns, understand public emotion trends, and make data-driven decisions based on regional language feedback. This method also helps highlight the emotional tone of the Tamil-speaking audience ina simple and accessible format.

Additionally, separate visualizations for each topic offer a more detailed sentiment breakdown, allowing for precise comparisons across different subjects. An interactive dashboard could also be incorporated to filter data dynamically, enhancing user engagement and enabling deeper exploration of sentiment trends.

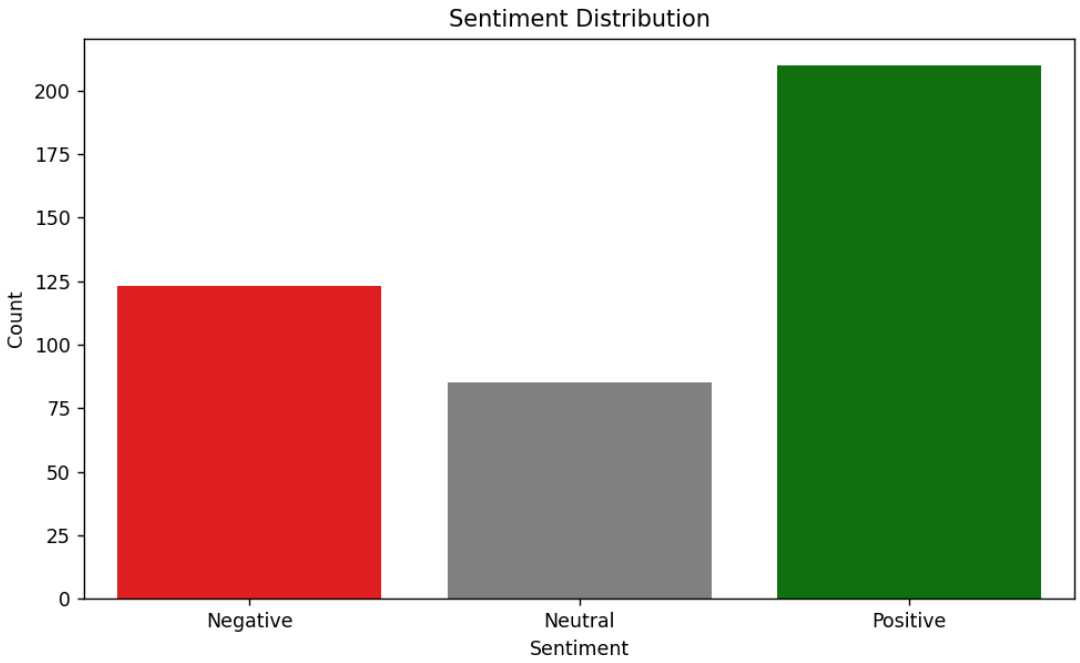


Figure 3.4 Screenshot of a bar plot

The Figure 3.4 the bar chart shows the distribution of sentiment in a dataset. The sentiment is categorized into positive, negative, and neutral .Positive sentiment is the most frequent, followed by negative sentiment, and neutral sentiment is the least frequent.

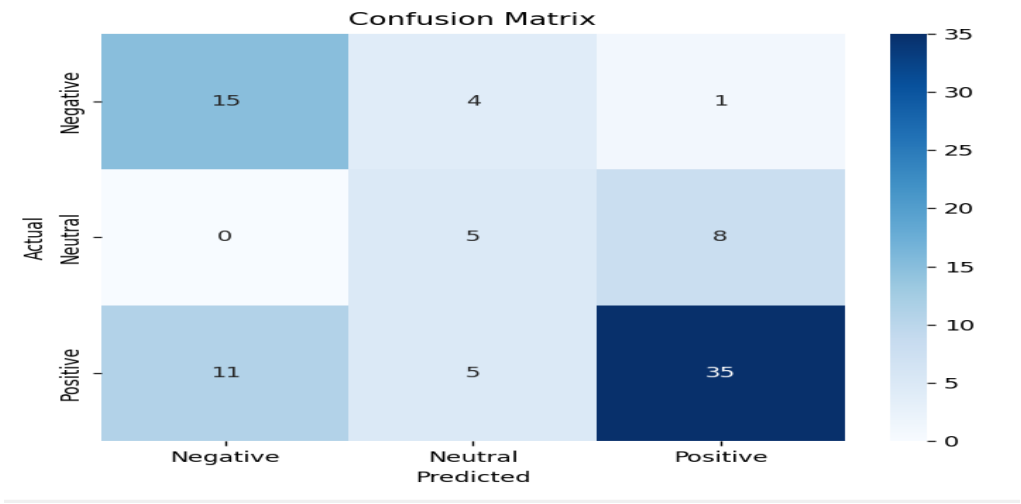


Figure 3.5 Screenshot of a Conclusion Matrix

Figure 3.5 shows the confusion matrix about the performance of a classification model, likely used for sentiment analysis. It compares the model's predictions (predicted labels) with the actual sentiment (true labels) in a test dataset. Each box in the matrix shows the number of data points.

3.3.2.6 Sentiment Classification

Sentiment classification is the process of identifying the emotional tone behind text comments positive, negative, or neutral using machine learning algorithms. In this project, classification is performed on Tamil regional language data, which includes dialectal variations, informal language, and code-mixed content. Among the models used, Support Vector Machine plays a key role. It is a statistical model that estimates the probability of a comment belonging to a specific sentiment category based on input features like word frequency and TF-IDF scores. The model is trained on labeled Tamil text data, enabling it to learn the relationship between words and sentiments. Once trained, it accurately classifies new, unseen comments. Support Vector Machine is preferred for its simplicity, interpretability, and solid performance in both binary and multi-class classification tasks. In the context of Tamil text, it helps identify how users feel about regional content, which can assist researchers and developers in understanding regional digital sentiment trends.

3.3.2.7 Sentiment Orientation

Sentiment orientation refers to the overall direction or polarity of sentiment positive, negative, or neutral expressed in Tamil comments, especially on platforms like social media or review sections. In this project, sentiment orientation helps capture the emotional feedback of Tamil-speaking users toward a particular topic, event, or digital content. By analyzing sentiment orientation, we gain valuable insights into regional opinions, especially in low-resource languages like Tamil, where traditional sentiment tools may fail. A positive orientation indicates appreciation or agreement, a negative orientation points to dissatisfaction or criticism, and neutral orientation shows balanced or indifferent responses. Identifying sentiment orientation in dialect-rich Tamil text helps in understanding cultural context, public mood, and can support decision-making in areas like digital communication, or social research.

3.4 TESTING

Testing is the process of executing a program with the intent of identifying and fixing errors. In our project, "Sentiment Analysis on Tamil Regional Language Using Machine Learning Algorithms," testing plays a crucial role in validating model accuracy, API responses, frontend functionality, and backend logic. The system is tested using various methods such as unit testing, integration testing, and functional testing.

3.4.1 Unit Testing

Unit testing involves testing each individual module of the project independently to verify correctness of logic and outputs. Below are the unit tests applied to the sentiment analysis system:

Table 3.1 Unit Test Cases for Preprocessing and Model

Test Case	Functionality Tested	Input	Expected Output	Test Result
Remove stopwords	Stopword removal	இந்தப் படம் சூப்பர்அனுபவமாகஇருந்தது	படம்சூப்பர்அனுபவமாகஇருந்தது	PASS
Tokenization	Token splitting	படம் சூப்பர் அனுபவமாகஇருந்தது	['படம்', 'சூப்பர்', 'அனுபவமாக', 'இருந்தது']	PASS
Vectorization	Word embedding	['படம்', 'சூப்பர்']	[0.23, 0.56, …]	PASS
Sentiment Prediction (Positive)	Sentiment classification	இந்தப் படம் சூப்பர்	Positive (Label: 1)	PASS
Sentiment Prediction negative	Sentiment classification	இது ஒரு போடமல்ல	Negative (Label: 0)	PASS
Sentiment	Sentiment	படம் சரியாய் பட்டமில்லை	Neutral (Label: 2)	PASS

Prediction (Neutral)	classification			

The Table 3.1 shows the preprocessing and sentiment prediction module details. This includes testing each step of the sentiment analysis process such as stopword removal, tokenization, vectorization, and sentiment classification. The table ensures that the input text is properly processed and correctly classified as Positive, Negative, or Neutral. Each function is verified individually to ensure the model behaves as expected for various Tamil language inputs.

3.4.2 Integration Testing

Integration testing ensures that all modules in the project work together seamlessly from input preprocessing to final sentiment prediction.

Table 3.2 Integration Test Cases for Full Pipeline

Test Case	Input	Expected Result	Test Result
User uploads comment	"இந்த படம் சூப்பர்"	Comment is received and displayed in frontend	PASS
Backend processes comment	Sent via API	Preprocessing functions are applied	PASS
Model receives cleaned input	"படம் சூப்பர்"	Model accepts and processes vectorized data	PASS
Sentiment response sent to frontend	Model prediction = 1	Frontend shows “Positive” response	PASS
Invalid input handling	Blank input or symbols	Appropriate error message displayed	PASS

The Table 3.2 shows how integrated components like the user interface, backend APIs, preprocessing logic, and model prediction work together. It ensures that the application flows without interruption and handles user actions

as expected.

3.4.3 Function Testing

Functional testing involves verifying that the software's functions behave according to the specified requirements or specifications. The goal is to validate the system against functional requirements, ensuring that each function of the software works as intended.

Table 3.3 Functional Test Cases

Test Case	Functionality Tested	Input	Expected Output	Test Result
Upload comment	Upload field in frontend	"இந்த படம் நல்லா இருந்தது"	Comment accepted	PASS
Predict sentiment	Click on "Predict" button	Clicked after input	Displays predicted sentiment	PASS
Display of prediction	UI result display section	Model returns 1	Shows “Positive”	PASS
Language handling	Tamil sentence processing	"இது ரொம்ப மோசம்"	Shows “Negative”	PASS

The Table 3.3 illustrates the functional flow of the application—starting from data input to result display. Each function, including comment input, preprocessing, prediction, and UI response, is tested to ensure user requirements are fulfilled.

CHAPTER 4

RESULT AND DISCUSSION

The previous chapter deals with the requirement analysis, design, implementation and testing of the project. This chapter deals with result of the project that may extend or meet some changes in the future.

4.1 RESULT

The Result of the project “SENTIMENT ANALYSIS ON TAMIL REGIONAL LANGUAGE USING MACHINE LEARNING ALGORITHMS” is obtained as expected. This project is implemented using Machine Learning Algorithms.

4.2 OUTPUT OF THIS PROJECT

This project provides a comprehensive sentiment analysis tool tailored to Tamil regional language content. A lightweight Flask-based web application was developed to allow users to input Tamil text and receive sentiment predictions instantly. The system was tested across devices — desktops, smartphones, and tablets — and the UI adapted well, providing a consistent and responsive experience.

Real-time user inputs were successfully processed by the backend model, and the results were displayed in clear sentiment labels (positive, negative, or neutral). Error handling was incorporated to catch empty inputs or unexpected characters, making the app more robust. The application includes the following modules:

- **Home Page**

The landing page provides a basic introduction to the project. It includes a clean navigation bar and buttons redirecting to other parts of the application.

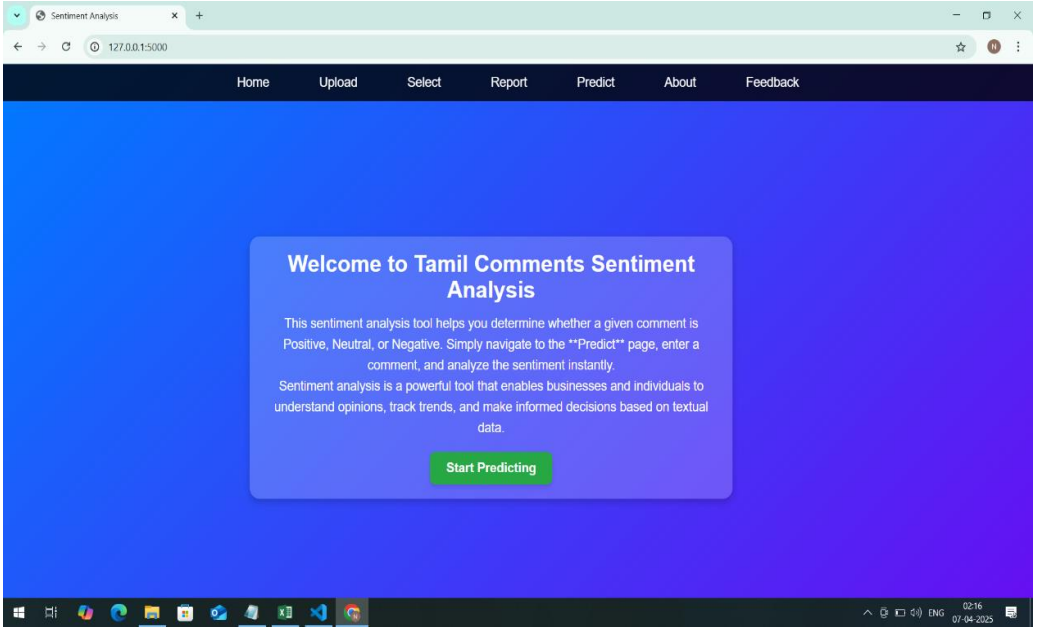


Figure 4.1 Screenshot of Home page

The Figure 4.1 Home Page acts as the landing page of the Tamil Comments Sentiment Analysis system. It provides a brief introduction about the project, explaining its purpose and capabilities. The interface is designed with a simple and clean layout, featuring a navigation bar at the top with menu options such as: upload, select, predict, report, about, feedback.

- **Upload Page**

The Figure 4.2 Upload Page allows users to upload their own dataset for sentiment analysis. It serves as the first step in processing user-generated data. The interface is simple and user-friendly, featuring a central upload panel with a clear instruction — “Upload Your Dataset”.

Users can select a CSV file from their local system using the “Choose File” button. Once the dataset is selected, clicking the “Upload” button initiates the process of storing the data in the system for further analysis.

This module ensures flexibility by allowing custom datasets, making the application adaptable for a variety of sentiment analysis tasks in the Tamil language.

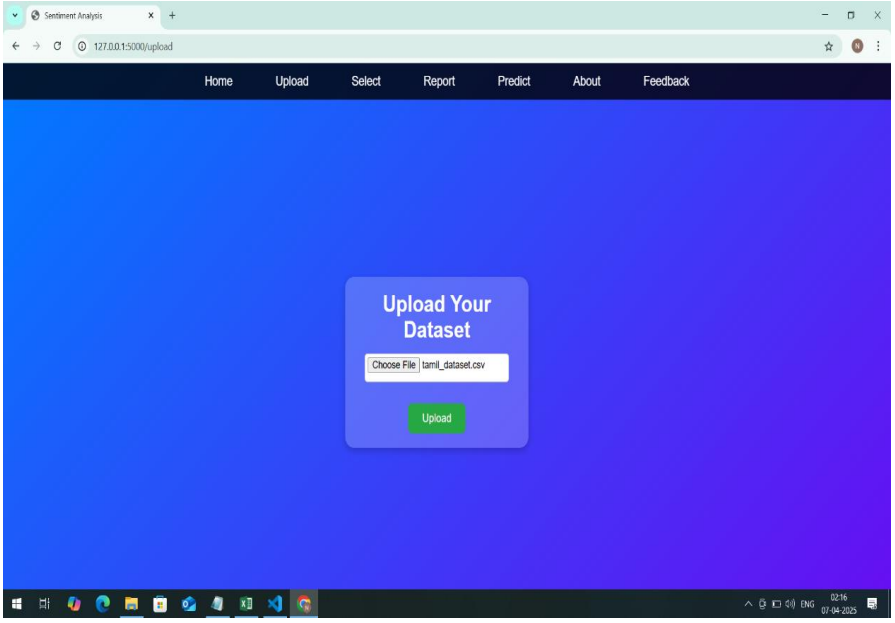


Figure 4.2 Screenshot of Upload page

- **Select Page**

The Figure 4.3 Select Comment Page is a key interactive module of the Tamil Comments Sentiment Analysis System. It provides users with the ability to view and select specific comments from the uploaded dataset. Once the dataset is uploaded, the page dynamically displays all the available Tamil comments in a dropdown menu. This allows users to easily scroll through the dataset and choose a particular comment that they want to analyze in detail.

This page is especially helpful in cases where the user wants to test the sentiment of individual comments rather than the entire dataset. This manual selection enhances the understanding of the system's working and helps in evaluating its prediction capabilities more transparently.

Moreover, this page works in close coordination with the vectorization and model selection options, ensuring that the chosen comment is processed using the selected algorithm and technique. It creates an ideal space for experimentation, making it easier for users to validate model behavior and gain insights into the accuracy and performance of various machine learning models when applied to Tamil language sentiment analysis.

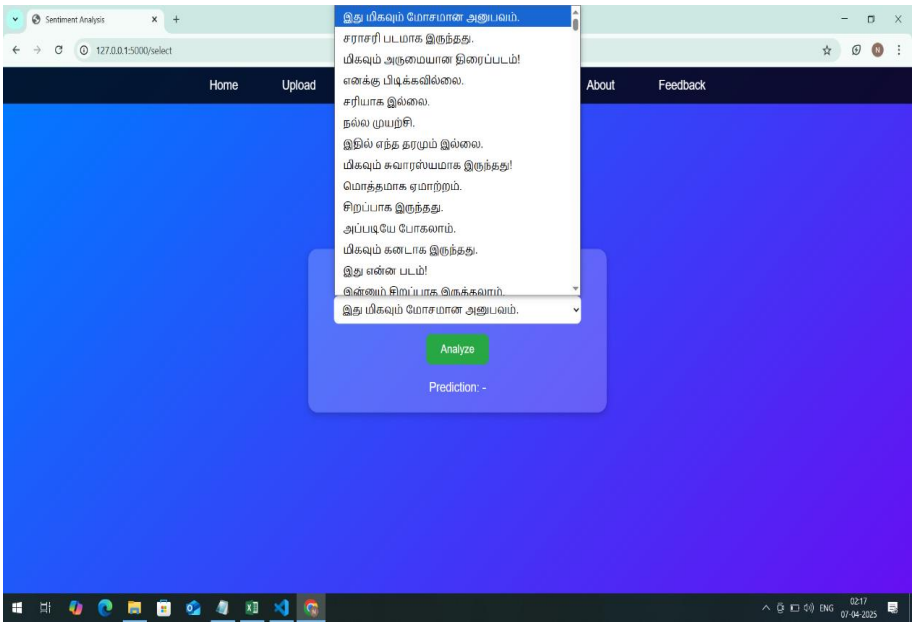


Figure 4.3 Screenshot of Select page

- **Report Page**

The Figure 4.4 Report page displays the uploaded dataset in a structured table format. It allows users to view all the Tamil comments along with their sentiment labels. The table helps in analyzing the data used for training and testing the model.

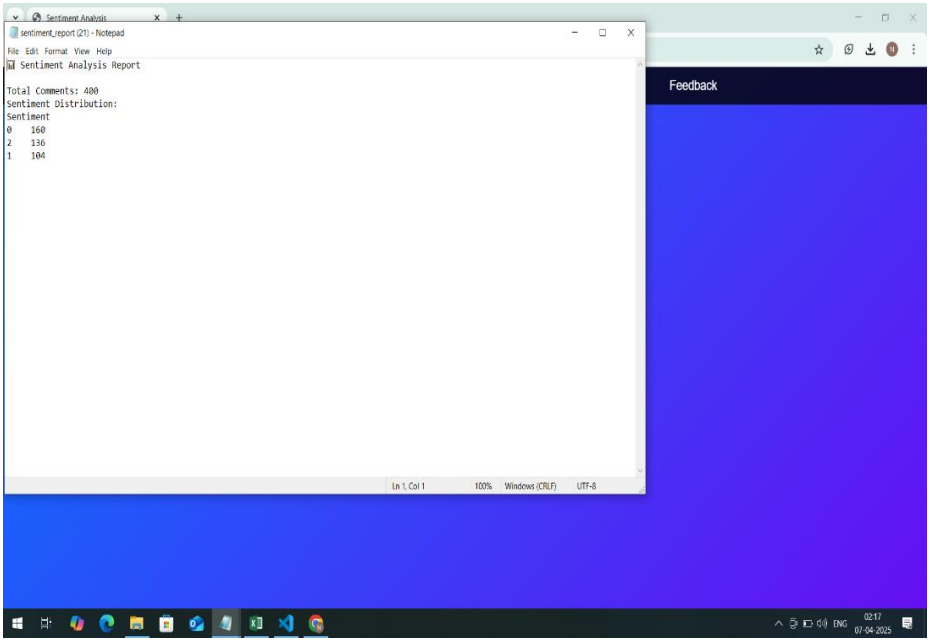


Figure 4.4 Screenshot of Report page

- **Prediction Page**

Users can input a Tamil comment in a text box and click the "Predict" button. The comment is sent via POST request to the Flask server, where it is preprocessed and classified. The predicted sentiment is displayed in real-time.

Users can input a Tamil comment in a text box and click the "Predict" button. The comment is sent via a POST request to the Flask server, where it is preprocessed using the saved TF-IDF vectorizer and then classified using the trained sentiment analysis model. The predicted sentiment—Positive, Negative, or Neutral—is displayed in real-time below the input box. This Figure 4.5 and Figure 4.6 Predict page helps users understand the sentiment of any Tamil comment instantly and interactively, making it user-friendly and efficient for real-time analysis.

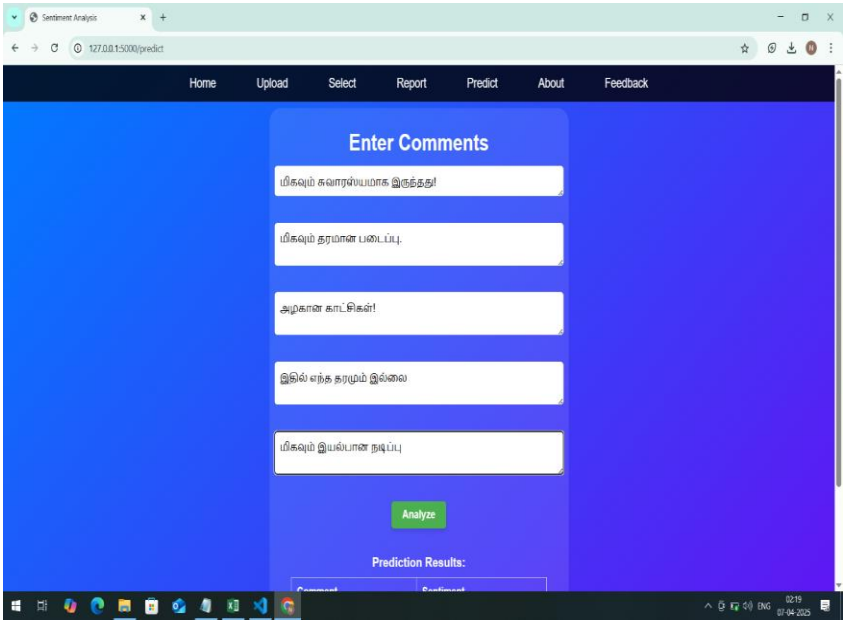


Figure 4.5 Screenshot of Prediction page

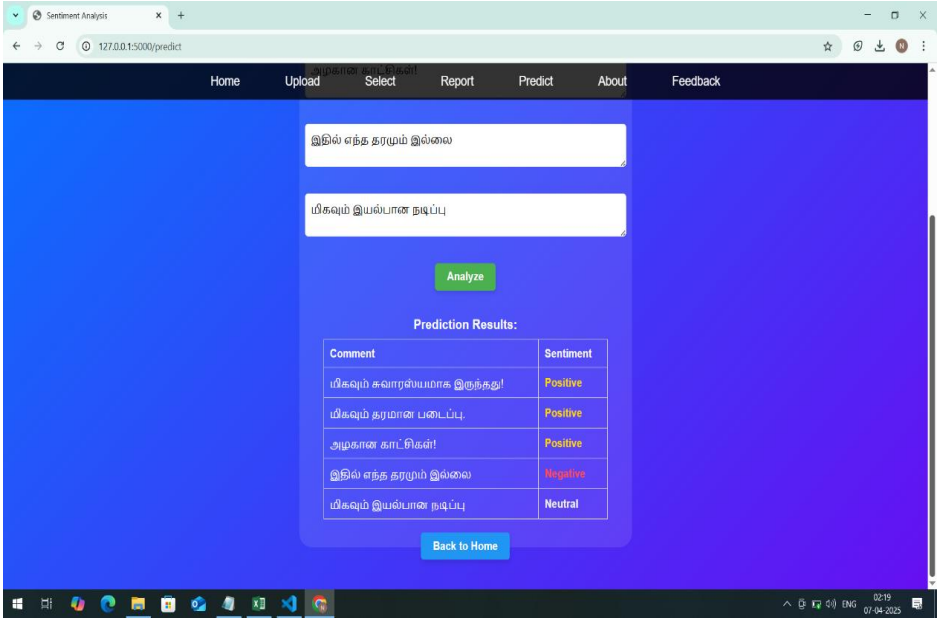


Figure 4.6 Screenshot of Prediction Analysis page

The Prediction Analysis Page (Figure 8.6) displays the sentiment prediction results based on user input. It visually presents the classified sentiment and confidence level using intuitive UI elements.

- **About Page**

The Figure 4.7 About page explains the background and objective of the project, including the motivation for building a Tamil sentiment analysis tool and its potential applications in social media monitoring and regional customer feedback the analysis.

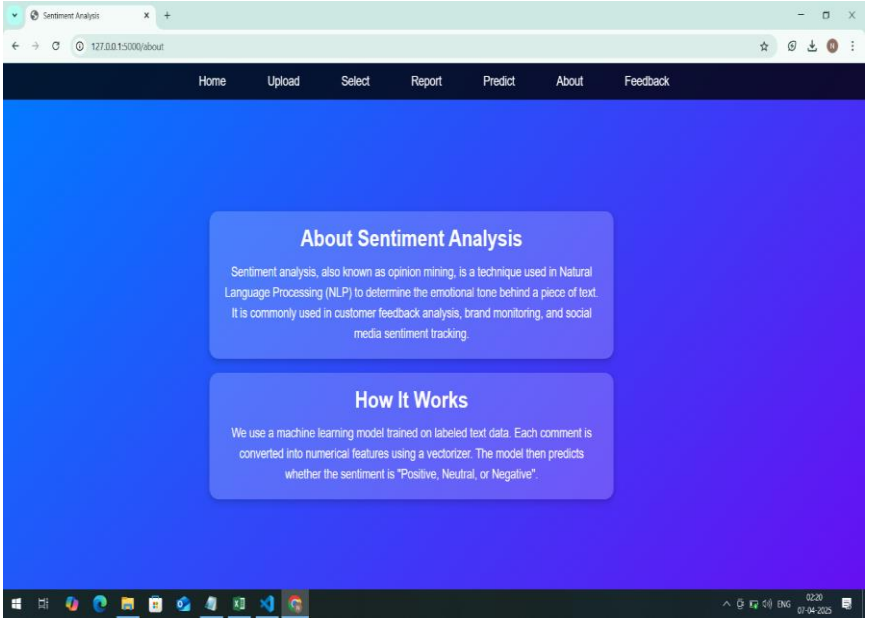


Figure 4.7 Screenshot of About page

- **Feedback Page**

The Figure 4.8 Feedback Page provides an essential communication channel between the users and the developers of the Tamil Comments Sentiment Analysis System. It allows users to share their thoughts, experiences, and suggestions directly through the application interface. The page features a simple text box where users can type their feedback and submit it with a single click.

Once submitted, the feedback is automatically stored in a dedicated CSV or text file, ensuring that all user responses are preserved for future reference. This data plays a crucial role in evaluating the effectiveness of the system, identifying bugs or issues, and gathering insights for continuous improvement. It helps developers analyze user behavior, preferences, and expectations, thereby enabling strategic enhancement planning and system upgrades.

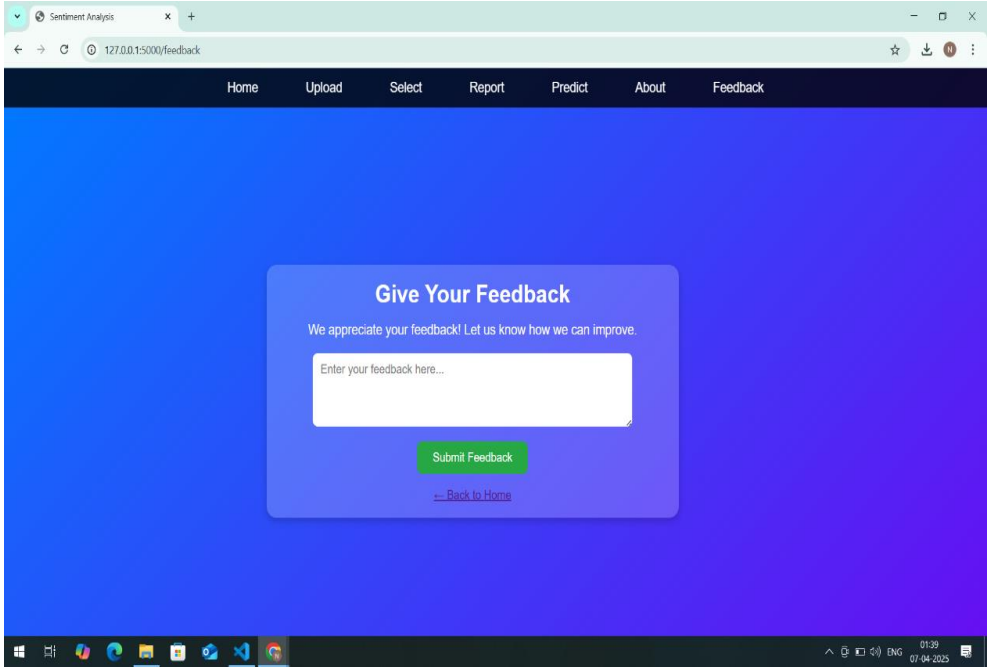


Figure 4.8 Screenshot of Feedback page

By encouraging active user participation, this page promotes a collaborative development environment where users contribute directly to refining the system's accuracy, usability, and performance. It also serves as a valuable repository of qualitative data that can inform the next phases of development and research.

CHAPTER 5

CONCLUSION AND FUTURE ENHANCEMENTS

5.1 CONCLUSION

This project demonstrates the implementation of a sentiment analysis tool tailored specifically to Tamil regional languages using machine learning algorithms. The primary aim was to develop a system that can accurately predict the sentiment (positive, negative, or neutral) of Tamil text comments, which can be applied in various domains like social media analysis, customer feedback, and more. The following conclusions can be drawn from the project:

- Language-Specific Adaptation: The project successfully adapted machine learning models for sentiment analysis to the specific challenges posed by the Tamil language, accounting for its unique structure, syntax, and regional variations.
- Improved Sentiment Accuracy: By leveraging state-of-the-art machine learning techniques like Natural Language Processing (NLP) and vectorization, the system showed promising accuracy in predicting sentiments for Tamil text inputs.
- Preprocessing and Text Cleaning: The preprocessing steps, including tokenization, stopword removal, and vectorization, were crucial in preparing the data for machine learning algorithms, ensuring optimal input for sentiment prediction.
- User-Friendliness: The front-end interface offers a seamless experience for users to input comments and receive sentiment analysis results instantly. This provides valuable insights into public opinion on various topics in Tamil, aiding decision-making processes in diverse sectors like entertainment, politics, and customer service.
- Real-Time Sentiment Feedback: The system's ability to analyze comments in real-time offers practical applications for analyzing public sentiment on social media, product reviews, and even customer service feedback, making it an effective tool for businesses and researchers alike.

5.2 FUTURE ENHANCEMENTS

While the current version of the sentiment analysis system for Tamil language comments is effective, there are several areas where future enhancements could further improve its functionality and scope. Some potential future developments include:

- Enhanced Model Training: Further model training with larger and more diverse datasets, including regional dialects and slang, can improve the accuracy of sentiment analysis across different Tamil-speaking communities. Incorporating additional data sources like news articles, social media comments, and forum discussions can help the model better understand contextual nuances in sentiment.
- Multilingual Support: Expanding the system to handle multiple languages within the Indian subcontinent could broaden its applicability. Integrating sentiment analysis for languages such as Hindi, Telugu, and Kannada alongside Tamil would create a more versatile tool.
- Emotion Detection: In addition to positive, negative, and neutral sentiment analysis, the system could be enhanced to identify emotions such as happiness, sadness, anger, or excitement in text. This can further enrich the insights provided to users, offering a more comprehensive understanding of user sentiment.
- Personalized Sentiment Feedback: Future versions could include a personalized sentiment feedback system where users can receive tailored recommendations based on the sentiment of their posts or comments. For instance, businesses could use this data to better understand customer reactions to specific products or services.
- Integration with Other AI Systems: The sentiment analysis tool could be integrated with chatbots or virtual assistants to provide automated, context-aware responses based on sentiment. For example, if a negative sentiment is detected, the system could trigger an automated customer service response.

REFERENCES

[1] S. H. Muhammad et al., *"AfriSenti: A Twitter Sentiment Analysis Benchmark for African Languages,"* Proc. Assoc. Comput. Linguistics (ACL), Nov. 2023.

[2] A. M. Alnasrawi, A. M. N. Alzubaidi, and A. A. Al-Moadhen, *"Improving sentiment analysis using text network features within different machine learning algorithms,"* Bull. Electr. Eng. Inform., vol. 13, no. 1, pp. 405–412, Feb. 2024.

[3] M. Rodríguez-Ibáñez, A. Casánez-Ventura, F. Castejón-Mateos, and P.-M. Cuenca-Jiménez, *"A review on sentiment analysis from social media platforms,"* Expert Syst. Appl., vol. 223, Art. no. 119862, Mar. 2023.

[4] W. Budiharto and Meiliana, *"Prediction and analysis of Indonesia Presidential election from Twitter using sentiment analysis,"* J. Big Data, vol. 5, no. 1, Art. no. 51, Aug. 2018.

[5] H. Taherdoost and M. Madanchian, *"Artificial Intelligence and Sentiment Analysis: A Review in Competitive Research,"* Computers, vol. 12, no. 2, Art. no. 37, Feb. 2023.

[6] A. Maruf, F. Khanam, M. M. Haque, Z. M. Jiyad, M. F. Mridha and Z. Aung, *"Challenges and Opportunities of Text-Based Emotion Detection: A Survey,"* IEEE Access, vol. 12, pp. 18415–18440, 2024.

[7] N. R. Yadav and A. Shukla, *"AI-Driven Sentiment Analysis in Social Media Content,"* International Journal for Research in Applied Science and Engineering Technology (IJRASET), vol. 11, no. 4, pp. 1234–1240, Apr. 2023.

[8] M. S. Khan and S. Jain, *"A Comparative Study of Sentiment Analysis Using Machine Learning Techniques,"* International Journal of Scientific Research in Computer Science, Engineering and Information Technology, vol. 8, no. 2, pp. 45–51, Mar.–Apr. 2023.

[9] A. Garg, A. Chaudhary, M. Goel and P. Rawal, *"Sentiment Analysis on Social Media Using Machine Learning and Deep Learning Techniques: A Review,"* International Journal of Innovative Research in Computer and Communication Engineering, vol. 11, no. 3, pp. 402–408, Mar. 2023.

[10] N. M. Patil and M. M. Khanapurkar, "*Text-Based Sentiment Analysis Using Deep Learning: A Comprehensive Review,"* International Journal of Engineering Research & Technology (IJERT), vol. 12, no. 5, pp. 150–157, May 2023.

APPENDIX

SAMPLE CODE

```
from flask import Flask, render_template, request, jsonify, redirect,
url_for, send_file

import pandas as pd

importjoblib

fromsklearn.metrics import classification_report

importos

app = Flask(__name__)

# Load Model and Vectorizer with Error Handling

try:

vectorizer = joblib.load("vectorizer.pkl")

model = joblib.load("sentiment_model.pkl")

except Exception as e:

print(f"Error loading model/vectorizer: {e}")

exit(1)

# Store uploaded data globally

uploaded_data = []

uploaded_df = None  # Store dataset for report generation

@app.route("/")

def home():

returnrender_template("home.html")

@app.route("/upload", methods=["GET", "POST"])

def upload():

globaluploaded_data, uploaded_df

ifrequest.method == "POST":

file = request.files.get("file")

if file:

try:
```

```
df = pd.read_csv(file)
if not {"Comment", "Sentiment"}.issubset(df.columns):
returnjsonify({"error": "Invalid file format. Ensure 'Comment' and 'Sentiment' columns exist."}), 400
uploaded_data = df["Comment"].dropna().tolist()
uploaded_df = df  # Store for report generation
return redirect(url_for("select"))
except Exception as e:
returnjsonify({"error": f"Error reading file: {str(e)}"}), 500
returnrender_template("upload.html")
@app.route("/select", methods=["GET", "POST"])
def select():
globaluploaded_data
if not uploaded_data:
return redirect(url_for("upload"))
ifrequest.method == "POST":
return redirect(url_for("predict"))
returnrender_template("select.html", comments=uploaded_data)
@app.route("/predict", methods=["GET", "POST"])
def predict():
ifrequest.method == "POST":
try:
data = request.get_json()
comments = data.get("comments", [])
if not comments:
returnjsonify({"error": "No comments provided"}), 400
comment_vectorized = vectorizer.transform(comments)
predictions = model.predict(comment_vectorized)
sentiment_labels = {0: "Negative", 1: "Neutral", 2: "Positive"}
```

```
results = []
for comment, pred in zip(comments, predictions):
sentiment_text = sentiment_labels.get(pred, "Unknown")
results.append({
                "comment": comment,
                "sentiment": sentiment_text
            })
returnjsonify({"predictions": results})
except Exception as e:
returnjsonify({"error": str(e)}), 500
returnrender_template("predict.html")
@app.route("/report", methods=["GET", "POST"])
def report():
globaluploaded_df
ifuploaded_df is None:
returnjsonify({"error": "No dataset uploaded"}), 400
    # Extract and clean data
df = uploaded_df.copy()
df.dropna(subset=["Comment", "Sentiment"], inplace=True)
    # Convert Sentiment to numeric safely
df["Sentiment"] = pd.to_numeric(df["Sentiment"],
errors="coerce").dropna().astype(int)
    # Ensure there are valid sentiment labels
ifdf.empty:
returnjsonify({"error": "No valid sentiment data available"}), 400
    # Vectorizing comments
    X = df["Comment"]
X_vectorized = vectorizer.transform(X)
```

```
 # Predicting sentiments
y_pred = model.predict(X_vectorized).astype(int)
    # Generate classification report
report = classification_report(df["Sentiment"], y_pred, output_dict=True)
    # Generate report text
report_text = "📊 Sentiment Analysis Report\n\n"
report_text += f"Total Comments: {len(df)}\n"
report_text += f"Sentiment
Distribution:\n{df['Sentiment'].value_counts().to_string()}\n\n"
    # report_text += "🔷 Model Performance\n"
    # report_text += f"Accuracy: {report['accuracy']:.2f}\n"
    # report_text += f"Precision: {report['weighted avg']['precision']:.2f}\n"
    # report_text += f"Recall: {report['weighted avg']['recall']:.2f}\n"
    # report_text += f"F1-score: {report['weighted avg']['f1-score']:.2f}\n\n"
    # Save report with UTF-8 encoding to avoid Unicode issues
report_file = "sentiment_report.txt"
with open(report_file, "w", encoding="utf-8") as file:
file.write(report_text)
returnsend_file(report_file, as_attachment=True)
@app.route("/feedback", methods=["GET", "POST"])
def feedback():
ifrequest.method == "POST":
feedback_text = request.form.get("feedback")
iffeedback_text:
with open("feedback.txt", "a", encoding="utf-8") as f:
f.write(feedback_text + "\n")
return redirect(url_for("home"))
returnrender_template("feedback.html")
```

```
@app.route("/about")
def about():
returnrender_template("about.html")
if __name__ == "__main__":
app.run(host="0.0.0.0", port=5000, debug=True)
```

Train_model.py

```
import pandas as pd
importjoblib
fromsklearn.feature_extraction.text import TfidfVectorizer
fromsklearn.model_selection import train_test_split
fromsklearn.linear_model import LogisticRegression
fromsklearn.naive_bayes import MultinomialNB
fromsklearn.svm import SVC
fromsklearn.ensemble import RandomForestClassifier
fromsklearn.metrics import accuracy_score, classification_report,
confusion_matrix
# Load Processed Dataset (After Feature Extraction)
file_path = r"C:\final year project85\back_end\tamil_dataset.csv"
df = pd.read_csv(file_path, encoding="utf-8")
# Ensure Sentiment Labels are Integers (-1, 0, 1)
df['Sentiment'] = df['Sentiment'].astype(int)
# TF-IDF Vectorization
vectorizer = TfidfVectorizer(max_features=5000, stop_words=None,
ngram_range=(1,2))
X = vectorizer.fit_transform(df['Comment'])
y = df['Sentiment']
# Train-Test Split (80% Training, 20% Testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

```
models = {
    "Logistic Regression": LogisticRegression(),
    "Naive Bayes": MultinomialNB(),
  "Support Vector Machine": SVC(),
    "Random Forest": RandomForestClassifier(class_weight="balanced")  }
# Train & Evaluate Models
best_model = None
best_accuracy = 0
for name, model in models.items():
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"\n{name} Accuracy: {accuracy:.2f}")
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
if accuracy >best_accuracy:
best_accuracy = accuracy
best_model = model  # Store Best Model
joblib.dump(best_model, "sentiment_model.pkl")
joblib.dump(vectorizer, "vectorizer.pkl")
print(f"\n🎯 Best Model Saved: {best_model.__class__.__name__} with
Accuracy: {best_accuracy:.2f}")
print("✅ Model &Vectorizer Saved Successfully!")
print("\nSentiment Distribution:")
print(f"Positive (2)  : {df['Sentiment'].value_counts().get(2, 0)}")
print(f"Negative (0) : {df['Sentiment'].value_counts().get(0, 0)}")
print(f"Neutral (1)   : {df['Sentiment'].value_counts().get(1, 0)}")
```

CERTIFICATIONS

CONFERENCE CERTIFICATION:

New Prince Shri Bhavani Arts and Science College, affiliated with the University of Madras and accredited with an 'A' grade by NAAC, successfully organized the International Conference on Contemporary Trends in Advanced Computing Technologies (ICCTACT) on 15th March 2025 in Chennai. Ms. G. Nandhini from B.S. Abdur Rahman Crescent Institute of Science and Technology participated in the conference and presented a research paper titled “Sentiment Analysis on Regional Language using Machine Learning Algorithms.” The conference, hosted by the Department of Computer Science & Computer Applications and IQAC, provided a vital platform for budding researchers, scholars, and academicians to exchange knowledge on advanced computing technologies and regional language processing using AI/ML models.

CONFERENCE PROCEEDINGS

SENTIMENT ANALYSIS ON REGIONAL LANGUAGE USING MACHINE LEARNING TECHNIQUES

G. Nandhini
Department Of Computer Applications,
B.S Abdur Rahman Crescent Institute Of Science And Technology,
Vandalur, Chennai - 600048
Email : nandhini2022gi@gmail.com
Dr. A. Jaya
Prof.,Department of Computer Applications,
B.S Abdur Rahman Crescent Of Institute Of Science And Technology,
Vandalur, Chennai – 600048
Email : jayavenkat2007@gmail.com

----------**ABSTRACT**----------

Nowadays, Social media has becoming an essential platform for people to share their opinions and emotions, to analyse the audience views a sentiment analysis plays vital role. The various entiment of the given statement is varying from Positive to Negative and Neutral sentiment. The comments are present in among various platforms, in that YouTube has become prominent one where people express their views based on the video content and quality. However, many users struggle to understand Regional Languages like Tamil language while analysis using machine learning algorithm. So in this research work analyse the Tamil text comments from social media platforms, where Tamil is spoken by approximately 80 million people worldwide. By applying sentiment analysis, we will understand the emotional polarity (positive, negative, or neutral) of Tamil text and the subjective expressions in urban discourse. For this, the tamil Comments are collected, preprocessed and labelled to classify the comments, and four machine learning models such as Support Vector Machine (SVM), Logistic Regression, Random Forest, and Naïve Bayes (NB) classifier is incorporated for classify the sentiment. The performance of these models is evaluated based on accuracy, precision, recall,f1-score and accuracy to determine the most effective approach for tamil sentiment analysis.

KEYWORDS – **Tamil sentiment Analysis, Natural Language Processing(NLP), Machine Learning.**

PROOF OF WORK

New Prince Shri Bhavani Arts and Science College

Affiliated to the University of Madras | Accredited by NAAC with 'A' grade | ISO 9001:2015 Certified

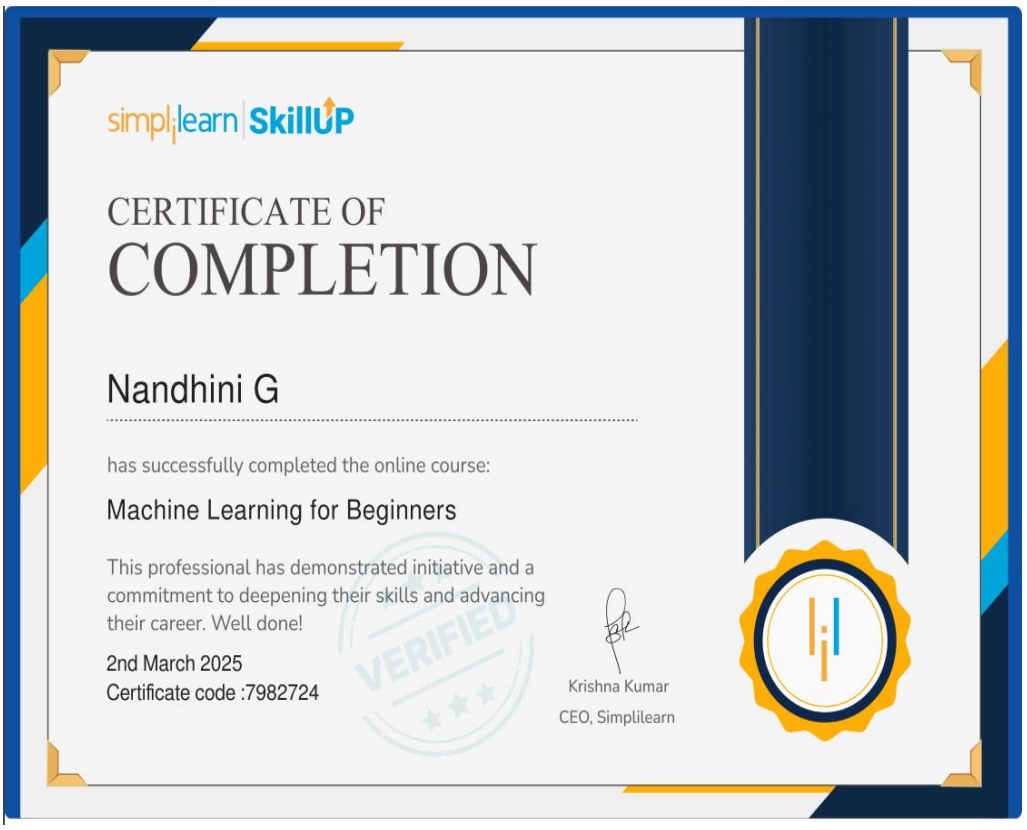
International Conference on "Contemporary Trends in Advanced Computing Technologies" (ICCTACT)

Certificate of Participation

This is to certify that Dr/ Mr/Ms G. NANDHINI of B S ABDUR RAHMAN CRESCENT INSTITUTE OF SCIENCE AND TECHNOLOGY has Participated / Presented a Paper Titled SENTIMENT ANALYSIS ON REGIONAL LANGUAGE USING MACHINE LEARNING ALGORITHMS in the International Conference on "Contemporary Trends in Advanced Computing Technologies" (ICCTACT) organized by Department of Computer Science & Computer Applications and IQAC, New Prince Shri Bhavani Arts and Science College, Chennai, on 15th March 2025.

Mrs.A.SIVARANI, Organizing Secretary | Dr.P.CALISTA BEBE, Convener | Dr.V.UMADEVI, Principal

TECHNICAL CERTIFICATION:



TECHNICAL BIOGRAPHY

Ms. Nandhini G. (RRN.230282601085), was born on 20th February 2003, in Chennai, Tamil Nadu. She has obtained the Bachelor of Computer Application Degree in April 2023 from Shrimati Devkunvar Nanalal Bhatt Vaishnav College For Women (Autonomous) . She is currently pursuing Master of Computer Applications Degree Program in B.S. Abdur Rahman Crescent Institute of Science and Technology, Vandalur, Chennai-48.

E-MAIL: nandhini2022gi@gmail.com

PHONE: 9360355129